

Technique	Efficiency	Application
Metropolis-Hastings	Moderate	General-purpose sampling
Gibbs Sampling	High	Bayesian inference
Hamiltonian Monte Carlo	Very High	High-dimensional models

Table 1: Comparison of MCMC Techniques

1 Introduction to MCMC Techniques

MCMC is a class of algorithms used to sample from complex probability distributions. It constructs a Markov chain whose equilibrium distribution matches the target distribution, allowing efficient sampling in high-dimensional spaces.

1.1 Types of common statistical Distributions

There are various MCMC techniques widely used These include:

- Bayesian statistics for posterior distribution estimation.
- Machine learning for probabilistic modeling.
- Finance for risk assessment.
- Genetics for evolutionary analysis.

2 Adaptive MCMC Methods

Adaptive MCMC modifies proposal distributions dynamically to improve sampling efficiency. **Examples of Adaptive MCMC**

- Adaptive Metropolis Algorithm – Adjusts proposal covariance matrix.
- Delayed Rejection Adaptive Metropolis (DRAM) – Enhances acceptance rates.
- Adaptive Gibbs Sampling – Adjusts transition probabilities

Python Example: Adaptive Metropolis Algorithm

```
import numpy as np
```

```
def metropolis_hastings(target_dist, proposal_std, steps):
```

```

samples = []
x = np.random.rand()
for _ in range(steps):
    x_new = x + np.random.normal(0, proposal_std)
    accept_prob = min(1, target_dist(x_new) / target_dist(x))
    if np.random.rand() < accept_prob:
        x = x_new
    samples.append(x)
return samples

target = lambda x: np.exp(-x**2) # Example target distribution
samples = metropolis_hastings(target, 0.5, 1000)

```

3 Efficiency of Various MCMC Techniques

3.1 Factors Affecting Efficiency

- Convergence Rate – How quickly the chain reaches equilibrium.
- Autocorrelation – Lower autocorrelation improves sampling independence.
- Computational Cost – Some methods require more resources.

3.2 Comparison of MCMC Techniques

Python Example: Gibbs Sampling

```

import numpy as np

def gibbs_sampling(steps):
    x, y = 0, 0
    samples = []
    for _ in range(steps):
        x = np.random.normal(y, 1)
        y = np.random.normal(x, 1)
        samples.append((x, y))
    return samples

samples = gibbs_sampling(1000)

```

4 Speed and Accuracy of MCMC

Trade-offs

- Higher accuracy often requires more iterations.
- Speed improvements can be achieved using parallelization.

Python Example: Gibbs Sampling

```
import numpy as np

def hamiltonian_monte_carlo(target, grad_target, steps, step_size):
    x = np.random.rand()
    samples = []
    for _ in range(steps):
        p = np.random.normal(0, 1)
        x_new = x + step_size * p
        accept_prob = min(1, target(x_new) / target(x))
        if np.random.rand() < accept_prob:
            x = x_new
        samples.append(x)
    return samples

target = lambda x: np.exp(-x**2)
grad_target = lambda x: -2*x
samples = hamiltonian_monte_carlo(target, grad_target, 1000, 0.1)
```